# Event-Log Augmentation under User-Defined Process Constraints

Massimiliano de Leoni[1] Alessandro Padella[1], Letizia Cimbro[1], and

University of Padua, Italy
letizia.cimbro@studenti.unipd.it alessandro.padella@unipd.it
deleoni@math.unipd.it

**Abstract.** Event logs produced by information systems are a key input for analyzing, monitoring, and improving process executions. However, such logs are often scarce or incomplete due to technical, managerial, or legal constraints, which limits the reliability of downstream analytics. Event-log augmentation can help produce larger and richer logs, but augmented traces must remain meaningful by satisfying process-level constraints. This paper introduces a constraint-aware technique for augmenting an initial event log with additional traces. The technique combines probabilistic models of observed process behavior with automaton-based representations of user-defined constraints. The goal is to generate traces that comply with the specified constraints while remaining behaviorally realistic and generalizing beyond the traces observed in the original log, thereby providing additional information for subsequent analyzes. We quantitatively assess the technique across five use cases, each assessed under increasingly stricter constraint sets derived from the original event log. The results show that our approach preserves compliance while increasing the diversity in the generated traces. We further evaluated one use case in the context of process discovery, comparing the fitness and precision of models mined from non-augmented logs, logs augmented without considering constraints, and logs augmented with our constraint-aware technique. The results indicate that considering constraints during augmentation leads to discovered models that better balance fitness and precision.

**Keywords:** Event-log augmentation · Generative methods · Process mining · User-defined constraints · Finite State Automata

## 1 Introduction

Process Mining seeks to understand and improve processes. It relies on an event log that records how individual process executions unfold [2]. However, as noted by Zimmermann et al. [16], data availability remains a major challenge for practitioners and researchers. Many techniques use AI-based models that require large training datasets, so they are ineffective when the event log is small. This triggers the need of event-log augmentation methods that enrich the set of the original traces with newly generated ones.

The problem of event-log augmentation has recently gained momentum. The objective is to generate new traces from those in the original event log that generalize with respect to the original event log but that, at the same time, show behavior that adheres

to that observed in the original log. The previous body of research does not support settings in which process analysts would like to impose constraints when generating new traces such as interaction protocols or compliance policies. The enforcement of domain-specific constraints inhibits the generation of unrealistic or semantically invalid behavior, which could otherwise adversely impact the effectiveness of subsequent downstream tasks. For example, if an event log is augmented without the imposition of such constraints, the resulting artificial traces may encode nonsensical process executions that could be incorporated into process models discovered from the logs.

This paper provides a technique to augment event logs under process' constraints that is based on finite-state automata (FSA) theory. In a nutshell, process constraints are given as a set of finite-state automata. The starting point is an event log $\mathcal{L}$ that needs to be augmented with additional traces. The event log $\mathcal{L}$ is then used to synthesize a stochastic finite-state automaton to represent the behavior observed in $\mathcal{L}$. This stochastic finite-state automaton is intersected with the provided set of constraint automata, thus obtaining a new stochastic constrained finite-state automaton that is only capable of generating traces compliant with the constraints.

Experiments were conducted with five use cases and three sets of constraints per case that were progressively restrictive. Our stochastic constrained automaton–based method was evaluated against three baseline approaches: (i) a baseline in which non-compliant traces are removed prior to the construction of a stochastic finite-state automaton that does not explicitly enforce the constraints, (ii) a state-of-the-art technique grounded in Conditional Variational Autoencoders [9], and (iii) a state-of-the-art approach that generates an event log from the set of mined constraints [3]. The experimental results show that our technique guaranties a higher level of generalization while maintaining similar adherence to the original event log (i.e., the log before augmentation) in most of the use cases. It should be emphasized that the first two baselines discussed above do not provide formal guarantees of constraint satisfaction (see also the discussion at the end of Section 5.3). Following its quantitative assessment with respect to generalization and conformance, we additionally apply our technique to three representative use cases to demonstrate its practical utility. In particular, we show that, by enforcing semantically meaningful constraints, the augmentation procedure yields models of superior quality, as reflected by higher F-score values between fitness and precision.

The remainder of this paper is structured as follows. Section 2 reviews the existing literature on event-log augmentation. Section 3 presents the preliminary concepts required to introduce our constraint-aware technique, which is then described in detail in Section 4. Section 5 presents the use cases, the metrics adopted for quantitative evaluation, the experimental setup, and the obtained results. Finally, Section 6 concludes the paper by outlining its limitations and suggesting directions for future research.

## 2 Related Work

Conventional data augmentation techniques frequently rely on the assumption of stochastic independence between samples [13]. However, this assumption is invalid in the con-

text of event logs, where events are intrinsically organized into sequences that exhibit pronounced temporal ordering and causal interdependencies between them [2].

Käppel et al. have proposed a first approach to event log augmentation in [10], where the event log augmentation framework relies on random noise-based transformations, while van Straten et al. [15] based it on replacement of process activities. However, these approaches do not enable feeding process constraints in, and consequently do not guaranty the compliance of the generated traces to them.

Graziosi et al. [9] propose a method for event-log generation that is based on Conditional Variational Autoencoders (CVAE). They rely on flagging the original traces as "regular" or "deviant", guiding the generation of "regular" traces, only. The set of constraints is not directly provided when training the CVAE model; instead, the model is expected to infer them by contrasting the "regular" and "deviant" behavior. As a result, there is no guarantee it will learn the exact constraints: the experimental results in Section 5.3 show that many generated traces violate them, especially when the process constraint set is larger.

In parallel, a growing body of work has emerged around Business Process Simulation (BPS) (see, e.g., [11]), which can be used to generate traces. However, they require a simulation model, which can certainly be discovered from event logs [2]. However, the discovery aims to ensure human readability as one of the main goals as this is necessary to test alternative what-if scenarios. Ensuring human readability often requires a modeling language that is sufficiently simple to interpret, which introduces representational biases that limit the behavior that can be observed and generated. These models can certainly be discovered from event logs, but this generation aims to ensure human readability as one of the main goals to enable testing alternative what-if scenarios.

The generation of constraint-compliant traces requires a method to enforce process constraints. To this extent, the DECLARE constraint-based process modeling language [3] provides a viable option. Di Ciccio et al. [3] introduced efficient discovery algorithms for DECLARE constraints from event logs, also capable of generating traces from a mined DECLARE model. However, they do not support the provision of additional constraints, and the generation depends only on the set of mined constraints. More recently, a data-aware extension of declarative process mining that generates both control-flow and data perspectives, has been proposed in [8], enabling more controlled and reproducible experimental setups. Nevertheless, the underlying generation mechanism remains solely driven by mined declarative constraints rather than by the statistical or structural properties of the event log itself.

## 3 Preliminaries

The starting point for a process mining-based system is an *event log*. An event log is a multiset of *traces*. Since this paper only focuses on the activity labels when traces are generated, each trace is abstracted as a sequence of activities, each describing a particular execution of a *process instance*:

**Definition 1 (Traces & Event Logs).** *Let $\mathcal{A}$ be the set of possible activities, and let $\mathcal{B}(\mathcal{A}^*)$ be the set of all multisets of sequences in $\mathcal{A}$. A trace $\sigma \in \mathcal{A}^*$ is defined as a finite*

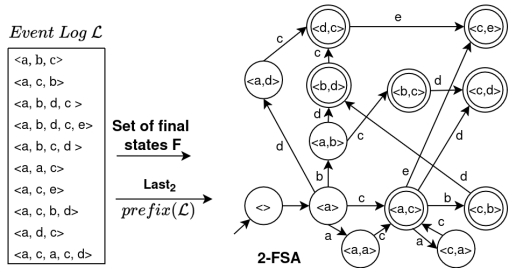



Fig. 1: Example of generation of a 2-FSA leveraging an event log $\mathcal{L}$ and a $last_2$ function.

*sequence of activities. An event log $\mathcal{L} \in \mathcal{B}(\mathcal{A}^*)$ is a multiset of sequences of activities in $\mathcal{A}$.*

To capture the different states in which a non-completed trace can be, we use the *prefixes*, i.e., a sequence of events that represent the process execution up to a certain point. Given a trace $\sigma = \langle a_1, \ldots, a_n \rangle$, the set of possible prefixes is defined as: $prefix(\sigma) = \{\langle\rangle, \langle a_1 \rangle, \langle a_1, a_2 \rangle, \ldots, \langle a_1, \ldots, a_n \rangle\}$. Analogously, we are also interested in annotating the set of prefixes of the traces in a given event log. The multiset containing all the prefixes of all the traces in an event log will be referred to as $prefix(\mathcal{L})$, i.e. $prefix(\mathcal{L}) = \uplus_{\sigma \in \mathcal{L}} prefix(\sigma)$.[1]

The remainder of this section illustrates the formal foundation of the event-log augmentation framework based on a stochastic finite-state automaton. The formalization relies on a helper function $last_k$ with $k \in \mathbb{N}$. Given a trace $\sigma = \langle a_1, \ldots, a_n \rangle$, $last_k(\sigma)$ returns the sub-trace consisting of the last $k$ events $\langle a_{n-k}, \ldots, a_n \rangle$; if $n < k$, the same trace $\sigma$ is returned. We can now define the concept of k-Finite-state Automaton:

**Definition 2 (k-Finite-state Automaton ($k$-FSA)).** *Let $\mathcal{L}$ be an event log defined over a set $\mathcal{A}$ of activities. Let $k$ be a natural number. The $k$-Finite Automaton is defined as $(S, \mathcal{A}, \delta, \langle\rangle, F)$, which denotes a finite state automaton, consisting of:*

- *a state set $S = \bigcup_{\sigma \in prefix(\mathcal{L})} last_k(\sigma)$,*
- *a transition set $\mathcal{A}$,*
- *a transition function $\delta : S \times \mathcal{A} \nrightarrow S$ s.t.*
  - $\forall a \in \mathcal{A},\ \langle a \rangle \in prefix(\mathcal{L}) \Rightarrow \delta(\langle\rangle, a) = \langle a \rangle$
  - $\forall \langle a_1, \ldots, a_n, a_{n+1} \rangle \in prefix(\mathcal{L})$.,
    $\delta(last_k(\langle a_1, \ldots, a_n \rangle), a_{n+1}) = last_k(\langle a_1, \ldots, a_{n+1} \rangle)$
- *the empty trace $\langle\rangle$ is the initial state,*
- *the set $F = \bigcup_{\sigma \in \mathcal{L}} last_k(\sigma)$ of final states.*

The following example illustrates how a 2-FSA is created starting from an event log:

[1] Symbol $\uplus$ indicates the union of multisets.

*Example 1 (2-FSA).* An example of 2-FSA is shown in Figure 1. Starting from an event log $\mathcal{L}$ composed of 10 complete traces, the multiset of prefixes $prefix(\mathcal{L})$ and the function $last_k$ are leveraged to infer the set of states $S = \{\langle\rangle, \langle a\rangle, \langle a, b\rangle, \langle a, a\rangle, \langle a, c\rangle, \langle a, d\rangle, \langle b, c\rangle, \langle b, d\rangle, \langle c, b\rangle, \langle c, e\rangle, \langle d, c\rangle, \langle c, d\rangle, \langle c, a\rangle\}$. The set of final states $F \subset S$ is then created from $\mathcal{L}$ and reported in figure with a double circle.

The transitions of a $k$-FSA can be annotated with the probability of occurrence, thus producing a Stochastic $k$-FSA:[2]

**Definition 3 (Stochastic k-Finite-state Automaton (Stochastic $k$-FSA)).** *Let $\mathcal{L}$ be an event log defined over a set $\mathcal{A}$ of activities. Let $k$ be a natural number. A Stochastic $k$-Finite-state Automaton is a tuple $(S, \mathcal{A}, \delta, \langle\rangle, F, P)$, where $(S, \mathcal{A}, \delta, \langle\rangle, F)$ is a $k$-FSA and $P : S \times \mathcal{A} \not\to [0, 1]$ is a function that for each $(s, a) \in \mathrm{dom}(\delta)$, returns the probability $P(s, a)$ of firing transition $(s, a)$.*

Note that for any final states $s \in F$, $0 \leq \sum_{a \in \mathcal{A}} P(s, a) \leq 1$. This occurs because trace generation stops when $s$ is reached with probability $1 - \sum_{a \in \mathcal{A}} P(s, a)$ and continues with probability $\sum_{a \in \mathcal{A}} P(s, a)$.

Leveraging the defined probability function $P$, the Stochastic $k$-FSA can be traversed to stochastically generate sequences of symbols that conform to the behavior captured by the underlying finite-state automaton. Consequently, each generated sequence is guaranteed to be valid with respect to the automaton structure, while the stochastic nature of the transitions enables the realistic modeling and simulation of process variability observed. The probabilities are derived directly from the original event log, ensuring that the resulting model accurately reflects the empirical control-flow dynamics observed in the real data.

Given an event log $\mathcal{L}$ and its corresponding $k$-FSA $(S, \mathcal{A}, \delta, \langle\rangle, F)$, the probability function of the corresponding Stochastic $k$-FSA $(S, \mathcal{A}, \delta, \langle\rangle, F, P)$ is defined as follows. For each $(s, a) \in \mathrm{dom}(\delta)$, the probability $P(s, a)$ is the ratio between *(i)* the number of prefixes that end with sequence $s$ followed by $a$ and *(ii)* the number of prefixes in which the trace suffix is $s$:[3]

$$P(s, a) \coloneqq \frac{\left|\{\sigma' \in \mathrm{prefix}(\mathcal{L}) : \mathrm{last}_{k+1}(\sigma') = s \oplus \langle a\rangle\}\right|}{\left|\{\sigma' \in \mathrm{prefix}(\mathcal{L}) : \mathrm{last}_{k}(\sigma') = s\}\right|} \tag{1}$$

Probabilistic transitions in the automaton are directly grounded in the observed frequency of activity successions in the event log. This data-driven probability assignment allows the automaton to generate new traces that not only comply with the structural rules encoded in the $k$-FSA but also preserve the stochastic properties of the original process behavior.

Let $\mathcal{Z} = (S, \mathcal{A}, \delta, q_0, F, P)$ be a Stochastic $k$-FSA. A trace $\sigma = \langle a_1, a_2, \cdots, a_n\rangle \in \mathcal{A}^*$ is admissible by $\mathcal{Z}$, and thus can be generated, iff there is a state sequence $s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \cdots \xrightarrow{a_n} s_n$ such that $s_i = \delta(s_{i-1}, a_i) \in S$ and $s_n \in F$ for all $i \in \{1, \ldots, n\}$. If $\sigma$ is admissible, the probability of generating $\sigma$ is $\prod_{i=1}^{n} P(s_{i-1}, a_i)$.

[2] Given a function $f$, in the remainder, $\mathrm{dom}(f)$ denotes the domain of $f$.

[3] Symbol $|\ |$ is overridden here to be applied to multisets, that is, to account for the sequence frequencies, and $\oplus$ is used to denote the concatenation of sequences.

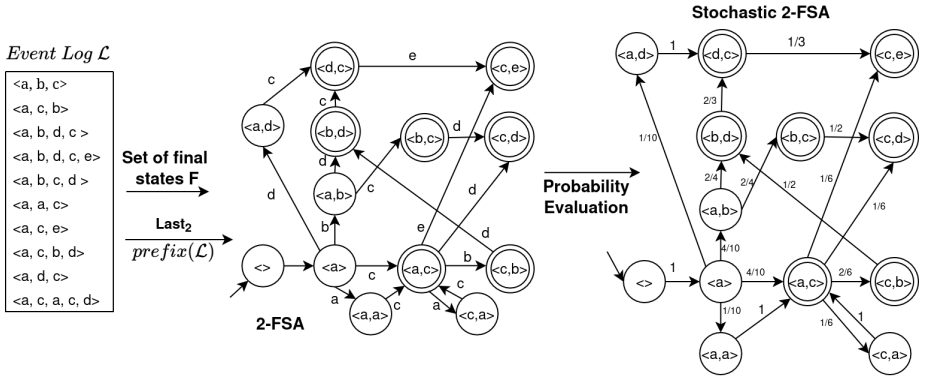



Fig. 2: Example of generation of a Stochastic 2-FSA, starting from the same event log proposed in Figure 1.

*Example 2 (Stochastic 2-FSA).* Figure 2 shows the Stochastic 2-FSA obtained from the 2-FSA in Figure 1. Using the multiset $prefix(\mathcal{L})$, transition probabilities are calculated according to the formula in Eq. 1. For example, in the state $\langle a \rangle$, the probability of firing the transition labeled $c$ is $\frac{4}{10}$. This is because the next state $\langle a, c \rangle$ appears 4 times right after $\langle a \rangle$, in the traces $\langle a, c, b \rangle$, $\langle a, c, e \rangle$, $\langle a, c, b, d \rangle$, and $\langle a, c, a, c, d \rangle$ out of 10 occurrences of the state $\langle a \rangle$, since every trace starts with $\langle a \rangle$. In contrast, consider the state $\langle d, c \rangle$, which is followed only once by $\langle c, e \rangle$ in the log. Since $\langle d, c \rangle$ occurs 3 times in total, and 2 of these are final states, the probability of transitioning with $e$ is $\frac{1}{3}$, while the probability of terminating generation at this state is $\frac{2}{3}$. The framework also supports looping behavior, as illustrated by state $\langle a, c \rangle$, which admits a transition labeled $a$ with probability $\frac{1}{6}$, leading to state $\langle c, a \rangle$ from which a return to $\langle a, c \rangle$ is guaranteed. Although such loops are structurally permitted, their occurrence remains statistically unlikely given the low associated transition probability.

## 4 The Framework Proposal

This section introduces the core framework for event-log augmentation under user-defined process constraints. These constraints are expected to be defined by process analysts in the form of finite-state automata:

**Definition 4 (Constraint Automaton).** *Let $\mathcal{A}$ be a finite set of activities. A Constraint Automaton over a set $\mathcal{A}$ of activities is a tuple $(Q, \mathcal{A}, \delta, q_0, F)$ where:*

- *$Q$ is a finite set of states,*
- *$\delta : Q \times \mathcal{A} \nrightarrow Q$ is the transition function,*
- *$q_0 \in Q$ is the initial state,*
- *$F \subseteq Q$ is the set of final states.*

Note that the above definition of constraint automata is compliant with every constraint definition supported by the DECLARE language [3, 7], although it is more general. Users

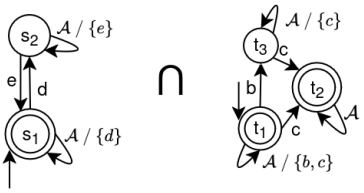



Fig. 3: Example of constraint automata: $C_1 = \{$if $d$ occurs, $e$ has to eventually occur after $d\}$, and $C_2 = \{$if $b$ occurs, $c$ has to occur before or after $b\}$

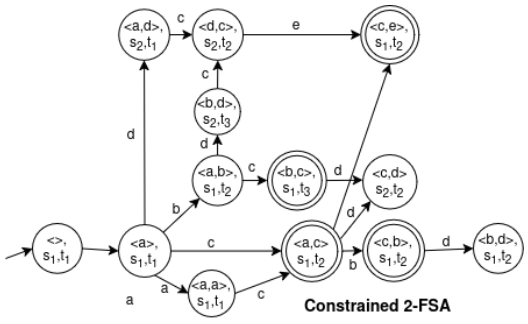



Fig. 4: Example of a Constrained 2-FSA.

can thus provide the constraints through the graphical notation of DECLARE. The following shows examples of constraint automata:

*Example 3 (Constraint Automata).* Figure 3 depicts two constraint automata, corresponding to two constraints: $C_1 = \{$if $d$ occurs, $e$ has to eventually occur after $d\}$, and $C_2 = \{$if $b$ occurs, $c$ has to occur before or after $b\}$. The automaton for $C_1$ consists of two states, $s_1$ and $s_2$, where only $s_1$ is accepting. In contrast, the automaton for $C_2$ has three states: $t_1$ represents when neither $b$ nor $c$ has occurred; $t_2$ represents the situation where $c$ has occurred (removing restrictions on $b$); and $t_3$ denotes that $b$ has occurred but $c$ has not yet appeared. The two automata will then be intersected to form a Constraint Automaton whose states are pairs drawn from the original automata's states. A state in this automaton is accepting if and only if both constituent states are accepting.

**Definition 5 (Constrained $k$-FSA).** *Let $\mathcal{A}$ be a set of activities, and let $\mathcal{C} = \{(Q_1, \mathcal{A}, \delta_1, q_{0,1}, F_1), \ldots, (Q_n, \mathcal{A}, \delta_n, q_{0,n}, F_n)\}$ be a set of constraint automata. The Constrained $k$-FSA is obtained by intersecting the Constraint Automata in $\mathcal{C}$ and a given $k$-FSA $(S, \mathcal{A}, \overline{\delta}, \langle\rangle, \overline{F})$ and consists in the automaton $(Q, \mathcal{A}, \delta, q_0, F)$ defined as follows:*

- *$Q = S \times Q_1 \times Q_2 \times \cdots \times Q_n$ is the product state space,*
- *$q_0 = (\langle\rangle, q_{0,1}, q_{0,2}, \ldots, q_{0,n})$ is the initial state,*
- *the transition function $\delta : Q \times \mathcal{A} \nrightarrow Q$ is defined as follows:*
  *For each state $(s, q_1, \ldots, q_n) \in Q$ and activity $a \in \mathcal{A}$, if $(s, a) \in \mathrm{dom}(\overline{\delta})$ and $(q_1, a) \in \mathrm{dom}(\delta_1)$ and $\ldots$ and $(q_n, a) \in \mathrm{dom}(\delta_n)$*

$$\delta\big((s, q_1, \ldots, q_n),\, a\big) = \big(\overline{\delta}(s, a), \delta_1(q_1, a),\ \delta_2(q_2, a),\ \ldots,\ \delta_n(q_n, a)\big),$$

- *the set of final states is $F = \overline{F} \times F_1 \times F_2 \times \cdots \times F_n$.*

*Example 4 (Constrained 2-FSA).* The Constraint Automaton log, obtained by intersecting the automata $C_1$ and $C_2$ from the Example in 3, is then intersected with the 2-FSA from the Example 1. Figure 4 shows the resulting Constrained 2-FSA where

states are within the Cartesian product of the states of the 2-FSA in Figure 1 and of $C_1$ and $C_2$. A tuple is a final state if and only if every component state from the input automata is final. In this case, the four identified final states are $(\langle c, e\rangle, s_1, t_2)$, $(\langle b, c\rangle, s_1, t_3)$, $(\langle a, c\rangle, s_1, t_2)$, and $(\langle c, b\rangle, s_1, t_2)$. Conversely, the states $(\langle c, d\rangle, s_2, t_2)$ and $(\langle b, d\rangle, s_1, t_2)$ have no possible outgoing transitions and are not final; such states are referred to as *dead ends*.

The use of a constrained $k$-FSA to augment an event log requires one to associate transitions with probabilities, yielding a Stochastic Constrained $k$-FSA:

**Definition 6 (Stochastic Constrained $k$-FSA).** *Let $\mathcal{A}$ be a set of activities, and let $\mathcal{C}$ be a set of constraint automata. Let $\mathcal{K}$ be a $k$-FSA, and let $\mathcal{S} = (\mathcal{K}, P)$ be an accordant Stochastic $k$-FSA. The Stochastic Constrained $k$-FSA of $\mathcal{C}$ and $\mathcal{S}$ is a tuple $(Q, \mathcal{A}, \delta, q_0, F, P)$ where $(Q, \mathcal{A}, \delta, q_0, F)$ is the Constrained $k$-FSA of $\mathcal{C}$ and $\mathcal{K}$ and $P : Q \times \mathcal{A} \not\to [0, 1]$ is a function that assigns a probability $P(q, a)$ to each pair $(q, a)$ in the domain of $\delta$.*

The function $P_c$ of a Stochastic Constrained Automata $(Q_c, \mathcal{A}, \delta_c, q_{0_c}, F_c, P_c)$ of a set $\mathcal{C}$ of constraint automata and a Stochastic $k$-FSA $(S, \mathcal{A}, \overline{\delta}, \langle\rangle, F, P)$ is computed as follows:

*1. Assignment of a zero probability to transitions reaching a dead state.* As also illustrated in Example 4, a Constrained $k$-FSA can contain some *dead* states $q$ from which no final state is reachable, namely there is no sequence $\langle b_1, \ldots, b_n\rangle \in \mathcal{A}^*$ such that $\delta(\ldots\delta(\delta(q, b_1), b_2)\ldots, b_n) \in F_c$. First, we identify the set $Q_d \subset (Q_c \setminus F_c)$ of *dead* states. For each transition $(q, a) \in \mathrm{dom}(\delta_c)$ such that $\delta_c(q, a) \in Q_d$, $P_c(q, a) \coloneqq 0$.

*2. Assignment of the probabilities to non-dead states.* In Step 1, the probabilities are set to zero for all transitions that lead to a dead state. To maintain consistency, the probabilities of the other transitions need to be scaled up: recalling that $Q_d$ identifies the dead states, for each transition $((s, \overline{q}), a) \in \mathrm{dom}(\delta_c)$ such that $\delta_c((s, \overline{q}), a) \notin Q_c$, its probability is set as follows:

$$P_c((s, \overline{q}), a) \coloneqq P(s, a) \cdot \frac{\sum_{a' \in \mathcal{A} \,:\, (s,a') \in \mathrm{dom}(\overline{\delta})} P(s, a')}{\sum_{a' \in \mathcal{A} \,:\, (s,a') \in \mathrm{dom}(\overline{\delta}),\ \delta_c((s,q),a') \notin Q_c} P(s, a')} \tag{2}$$

Note how the ratio present in Equation 2 is always larger than or equal to one. If the probability of any transition $((s, q), a')$ is set to zero, the ratio is larger than one, thus distributing those probabilities to every transition that enables reaching a final state, proportionally to the original probabilities.

*Example 5 (Stochastic Constrained 2-FSA).* In this example, also depicted in Figure 5, the Stochastic 2-FSA created in example 2 and the Constrained 2-FSA created in example 4 are used to create the Stochastic Constrained 2-FSA. As reported in Definition 6, its construction implies two steps: in the first step the transitions reaching a dead state are assigned probability 0. This was applied to the dead states $(\langle c, d\rangle, s_2, t_2)$ and $(\langle b, d\rangle, s_1, t_2)$ that have been assigned probability 0. Then, the probabilities are assigned and rescaled, since the transition from state $(\langle a, c\rangle, s_1, t_2)$ to $(\langle c, d\rangle, s_2, t_2)$ had probability $\frac{1}{6}$, now the probabilities of the other transition that can be fired after $(\langle a, c\rangle, s_1, t_2)$ are multiplied by a factor of $\frac{6}{5}$.

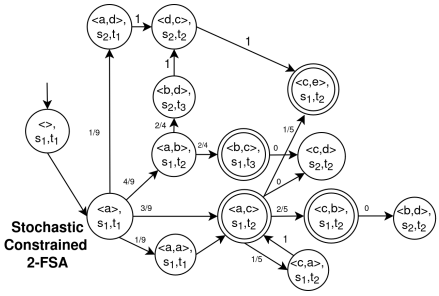



Fig. 5: Visual representation of the Stochastic Constrained 2-FSA, obtained from the Constrained-2FSA in Figure 4 and the Stochastic 2-FSA in Figure 2.

## 5 Quantitative Evaluation

The approach has been implemented in Python and is accessible in an anonymous repository [1].

This section details the experiments that have been carried out to quantitatively evaluate the quality of the generated event logs in terms of adherence to and generalization of the original event log.

### 5.1 Use Cases and Definition of Constraint Sets

The validity of the framework was assessed on five processes, for which event logs were available: *Purchasing*, a synthetic purchase-to-pay process provided with the Fluxicon Disco tool[4]; *Production*, a manufacturing production process (MP) exported from an Enterprise Resource Planning (ERP) system[5]; *Consulta*, the Academic Credentials Recognition (ACR) process of a Colombian University[6]; *Lending*, a synthetic loan application process from which a log has been generated[7]; and *Emergency Dept.*, a real process provided by an Italian hospital emergency department, which is private.

Since our controlled experiments aims to be objectively designed and run, the constraints are discovered from the event logs. The constraints are discovered through the Declare miner implemented in `pm4py`.[8] It is well known that each Declare constraint can be transformed into a corresponding finite state automaton as per Definition 4 that accepts exactly those traces that satisfy the constraint [6]. In particular, we mined the constraints with a support larger than 0.8 and Conditional-Probability Increment Ratio (CPIR) larger than 0.85. The choice is in line with the guidelines in [12]. For each use case, we then chose the top 5, 25 and 50 constraints in terms of support, aiming to

[4] `https://fluxicon.com/academic/material`

[5] `https://doi.org/10.4121/uuid:68726926-5ac5-4fab-b873-ee76ea412399`

[6] `https://zenodo.org/records/5734443`

[7] `https://zenodo.org/records/8059489`

[8] The `pm4py`'s implementation of the Declare miner is discussed at `https://processintelligence.solutions/pm4py/api/api/pm4py.algo.discovery.declare.algorithm.html`

illustrate whether our framework scales with larger and larger sets of constraints and whether the benefits were more or less evident with a larger set of constraints.

### 5.2 Experimental Settings

A set of experiments were run on the combinations of the use cases and sets of defined constraints that were mentioned above.

The ultimate goal is to illustrate that our framework based on Constrained Stochastic $k$-FSA is able to increase the level of generalization while maintaining a similar level of distance between the generated and the original event log. Typically the distance with the original event log and the generalization are opposite dimensions, but we aim to show that our framework enables to obtain a better trade-off, if compared with what state-of-the-art techniques are able to achieve.

*Experimental Methodology.* The event log $\mathcal{L}$ of each use case was divided into a training log $\mathcal{L}_{train}$ and a test log $\mathcal{L}_{test}$ after a temporal splitting. The training log $\mathcal{L}_{train}$ was constructed from the first 50% of the traces. For each use case, log $\mathcal{L}_{train}$ was used with each set $\mathcal{C}$ of the mined constraint (cf. previous paragraphs) to build our stochastic Constrained $k$-FSA where $k$ was set to vary between 1 and 10. The determination of the best $k$ was made as follows. For each set $\mathcal{C}$ of constraints, we retained the sub-log $\mathcal{L}_{test}^{C}$ of the trace in $\mathcal{L}_{test}$ that did not violate $\mathcal{C}$. For each $k \in \{1, \ldots, 10\}$, we then generated 10 synthetic event logs with the same number of traces as $\mathcal{L}_{test}^{C}$ and computed the average trace entropy of these 10 synthetic event logs and the average 2-gram log distance between these 10 synthetic event logs and $\mathcal{L}_{test}^{C}$. For each use case and set $\mathcal{C}$, we ultimately retained the Stochastic Constrained $k$-FSA with the best balance between entropy and log distance, computed on a validation log composed by the 10% of the traces in $\mathcal{L}_{train}$, to avoid data leakage.

*Experimental Metrics.* The level of generalization was measured through the trace entropy computed in the event logs generated [4]. Trace entropy measures the diversity of traces in an event log. It is computed by considering the relative frequency of each distinct trace and applying the standard entropy formula for sequences. A low trace entropy indicates that the log contains mainly similar repeated traces, while a high trace entropy indicates greater variability among the traces [4]. The distance to the original event log was computed through the 2-gram log distance (2-gram), introduced in [5], between the generated event log and $\mathcal{L}_{test}^{\mathcal{C}}$, previously introduced as the sub-log of the traces of $\mathcal{L}_{test}$ that comply with the set $\mathcal{C}$ of constraints of interest in the experimental run. The 2-gram log distance compares two event logs by counting how often each pair of consecutive activities occurs on the traces of both logs. Then it computes the normalized sum of the absolute differences between these 2-gram frequencies. Alternatively, we could have adopted the Control-Flow Log Distance (CFLD) measure [5]. However, CFLD is computationally demanding in terms of both time and memory demands, because of which it could not be computed for several of the use cases considered in our experiments. For this reason, we relied on the 2-gram distance, which *"effectively captures the effect of perturbations along the control-flow perspective, achieving similar results as the Control-Flow Log Distance (CFLD) measure [...] while being much more computationally efficient"* [5].

*Comparison Baselines.* The synthetic event logs generated through our Stochastic Constrained $k$-FSA were compared with those obtained through three baselines from the state of the art:

1. A non-stochastic constrained $k$-FSA with same value for $k$ as used for our Stochastic Constrained $k$-FSA. In this setting, the traces that violate the constraints are filtered out before building the non-stochastic $k$-FSA.
2. The CVAE method by Graziosi et al. [9] flagging the non-conformant traces as "deviant" and the conformant traces as "regular".
3. The RuM toolkit by Alman et al. [3], where the Stochastic $k$-FSA is replaced by Declare model, which is mined from the event log. Namely, the constraints of the mined Declare model are intersected with the constraints provided by the user.

In addition to the above, we also considered our Stochastic Constrained $k$-FSA and a non-stochastic constrained $k$-FSA by setting $k = \infty$ as additional baselines. The expectation is that $k = \infty$ corresponds to the setting in which traces are simply sampled from the training logs, thereby resulting in the lowest levels of generalization.

### 5.3 Experimental Results

| Use Case | 5 constraints | | 25 constraints | | 50 constraints | |
|---|---|---|---|---|---|---|
| | S $k$-FSA | CVAE | S $k$-FSA | CVAE | S $k$-FSA | CVAE |
| Consulta | 0% | 12.7% | 6.2% | 17.2% | 9.0% | 76.4% |
| Production | 0.5% | 0% | 8.3% | 85.4% | 9.3% | 90.6% |
| Purchasing | 0% | 30.6% | 26.4% | 46.5% | 27.0% | 50.8% |
| Emergency Dept | 0% | 8.2% | 9.2% | 5.9% | 10.2% | 8.0% |
| Lending | 0% | 5.2% | 0% | 9.7% | 0% | 67.3% |

Table 1: Percentage of generated traces non-conformant to constraints by use case, scenario, and constraint setting. We only report for the Stochastic $k$-FSA with $k$ finite and optimized, and for CVAE because they are the only baselines that do not guarantee compliance in the generated traces. The reported values are the mean of 10 repetitions.

The first assessment focused on the baseline based on Stochastic $k$-FSA (S $k$-FSA) and CVAE, which do not provide a formal guaranty that the generated traces comply with the set of constraints. This is illustrated in Table 1, where, for each use case, the percentage of non-compliant traces with respect to the respective set of constraint is shown. The experiment on each use case and set of constraints was repeated 10 times to generate so many event logs. As expected, these two baselines can generate non-compliant traces, and this grows as the constraints are more and more restrictive. Experiments are shown that CVAE tends more than S $k$-FSA to generate non-compliant traces.

The numerical results are visualized in two graphs in Figure 6, where results are separately reported for each use case but they are aggregated by the different sets of constraints employed (namely the top 5, 25 and 50 constraints discovered through declare miner). These two comparative views simplify the analysis, compared with the detailed reporting of the raw values for each use case and set of constraints. The latter is provided in the appendix of the extended version of this paper in the repository [1].

Figure 6 reports the difference in the entropy or 2-gram values of each baseline, compared to our method based on stochastic constraint $k$ -FSA, normalized with respect to the value obtained by our method. Namely, if $x$ and $y$ are the values obtained

respectively by a baseline method and our method based on $k$-FSA, this normalized difference is therefore reported as $(x - y)/x$. For each use case and comparison baseline, the difference in entropy and 2-gram was computed separately for the different sets of constraints, then summarized by its median value, with the whiskers marking the minimum and maximum across those three sets of constraints. As mentioned in Section 5.2, $k$ was optimized and changed for each use case: $k$ was set to 3 for the use cases *Consulta*, *Purchasing* and *Emergency Department* and $k = 2$ for *Production* and *Lending*. RuM was excluded from both graphs since its values were consistently far worse outside the range of the other methods, compressing the differences with the other baselines into an unreadable scale.

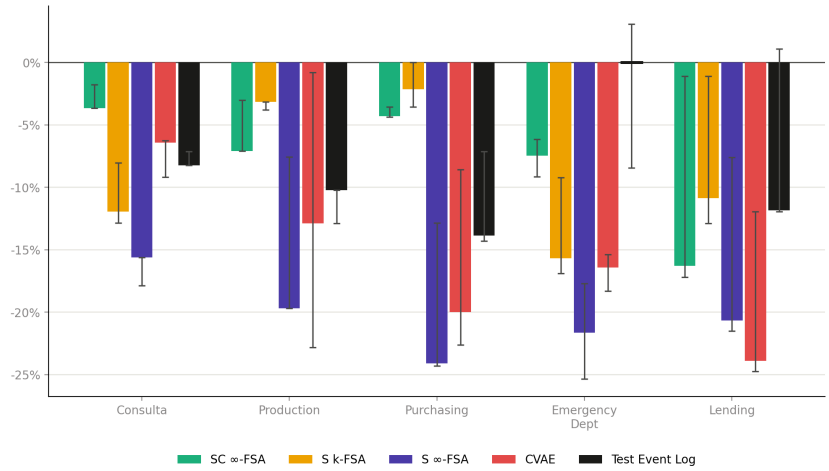
Figure 6a reports on the normalized differences in entropy between the baselines and our method based on $k$-FSA, per use case: negative values indicate a reduction in entropy of the baselines. Note that every method was used to generate the same number of traces as the test set for each configuration, ensuring comparability of the results. The differences are negative for every use case, meaning that the entropy of the event logs generated through our method based on Stochastic Constrained k-FSA tends to be higher, thus confirming our hypothesis that our method is better at generalizing when generating traces. As a further baseline for comparison, the normalized differences are also reported with respect to the entropy of the test log itself (see black bars): the negative difference with respect to the test event log illustrates that our method is capable of generalizing more than the original event log.

Figure 6b conversely focuses on the normalized difference of the 2-gram metrics between the baselines and our method based on Constrained $k$-FSA: a negative value indicates that the value of the 2-gram metrics of the corresponding baseline is lower than what the generation through our method achieves, and lower value indicates more adherence of the generated traces to those of the original log. The graph shows that CVAE is well above our method in every use case, while the other baselines generally track close, except in *Lending*, where all baselines give higher 2-gram values, and *Emergency Department*, where the baselines based on Stochastic non-Constrained FSA (S $k$-FSA and (S $\infty$-FSA) show lower 2-gram values.The *Emergency Department* use case warrants a separate discussion: its median 2-gram value is $17\%$ lower (i.e., better) than that of our method, whereas its entropy is $15.7\%$ lower (i.e., worse) than our method. Hence, our approach does not appear to outperform the baseline S $k$-FSA. A plausible explanation is that the underlying process shows limited variability, as indicated by the fact that the entropy of the test event log is essentially equal to that of the event log produced by our method, which also yields the best entropy among the baselines. Under these circumstances, adding constraints does not substantially affect the generation process, but it does decrease the similarity to the test event log. However, recall that the baseline based on Stochastic non-constrained $k$-FSA may generate traces that do not comply the sets of constraints: in fact, the percentage of generated, non-compliant traces for this baseline is between $5.9\%$ and $10.2\%$ in every experiment with this use case.

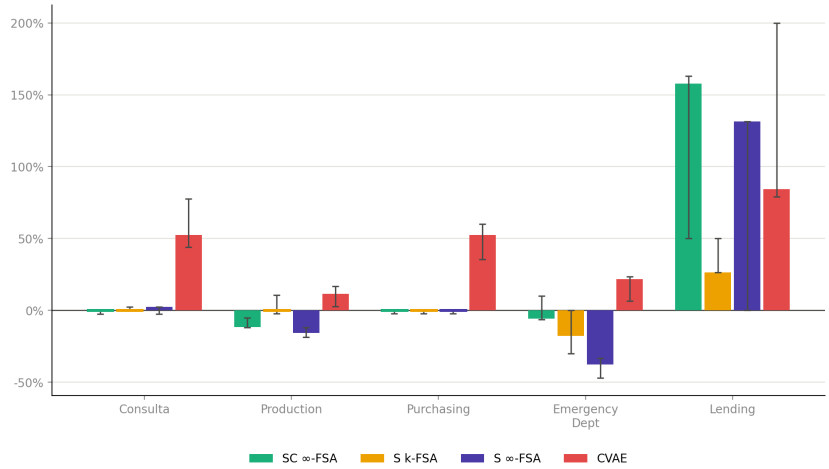
We conclude the assessment with an average of the computation time of our method based on SC $k$-FSA and the baselines. Table 2 reports our findings for

| Use Case | SC$k$-FSA | SC$\infty$-FSA | baseline $k$ | S $\infty$-FSA | CVAE | RuM |
|---|---|---|---|---|---|---|
| Consulta | 7min | 7min | 2min | 2min | 15min | 3min |
| Production | 5min | 5min | 1min | 1min | 70min | 2min |
| Purchasing | 6min | 6min | 1min | 1min | 25min | 2min |
| Lending | 22min | 22min | 11min | 11min | 1d 4h | 9min |
| Emergency Dept | 54min | 54min | 29min | 29min | 1d 9h | 15min |

Table 2: Computational time to train via our technique and generate ten event logs.




(a) Normalized entropy differences between the baseline approaches and our $k$-FSA–based method. Negative values denote a reduction in entropy for the baselines, which is undesirable. For reference, the normalized entropy of the test event log is also reported.




(b) 2-Gram differences between the baseline methods and our $k$-FSA–based method. Negative values indicate a decrease in entropy for the baseline methods, which is considered beneficial.

Fig. 6: Relative deviation from the SC $k$-FSA baseline (SC 3-FSA for Consulta, Purchasing, and Emergency Dept; SC 2-FSA for Production and Lending). Bars represent the median across configurations with 5, 25, and 50 constraints, while whiskers indicate the full range (minimum to maximum). RuM is omitted from the visualization due to its consistently large deviations.

each case study. CVAE has very high computational times, while

the others are characterized by times of the same order of magnitude. Notably, of the finite-state automata underlying the Stochastic Constrained $k$-FSA does not introduce any significant computational overhead, even when one needs to intersect 50 FSAs (namely, one per constraint).

## 6 Conclusions

This paper presents a novel constraint-aware framework for event-log augmentation, integrating user-defined process constraints, expressed as finite-state automata, directly into the generative process that is based on them. The approach ensures strict compliance of all generated traces with the specified constraints while preserving trace variability, which is essential for effective process mining tasks such as simulation and predictive monitoring. Empirical evaluation across five use cases demonstrates a superior compliance-variability trade-off compared to baselines that filter non-compliant traces, generate event-logs directly from constraints and to a state-of-the-art frameworks, as validated by entropy-based metrics and 2-gram log distance. In addition to its competitive performance and efficiency, the introduced framework is inherently white-box, making the integration of process constraints considerably simpler compared to black-box deep-learning approaches. The technique proposed here is naturally inapplicable if mutually conflicting constraints are included, yielding Stochastic Constrained FSA that accept no traces. However, techniques exist that can repair these sets of constraints, e.g. by Di Ciccio et al. [7].

Future work aims to evaluate alternatives built on neurosymbolic AI infrastructures [14]. However, it remains to be demonstrated that their potential advantages outweigh the computational overhead these infrastructures typically introduce. Future work also aims at the generation of additional perspectives on, e.g., data, resource, and time, under constraints that involve these perspectives. Furthermore, we plan to conduct some evaluation with stakeholders to further assess the practical usability of the method.

## References

1. Code and Data Repository of this paper. `https://anonymous.4open.science/r/Constrained_data_aug-6B86`
2. van der Aalst, W.M.P.: Process Mining: Data Science in Action. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
3. Alman, A., Di Ciccio, C., Maggi, F.M., Montali, M., van der Aa, H.: Rum: Declarative process mining, distilled. In: Business Process Management. pp. 23–29. Springer International Publishing, Cham (2021)
4. Back, C.O., Debois, S., Slaats, T.: Entropy as a measure of log variability. J. Data Semant. **8**(2), 129–156 (2019)
5. Chapela-Campa, D., Benchekroun, I., Baron, O., Dumas, M., Krass, D., Senderovich, A.: Can I trust my simulation model? measuring the quality of business process simulation models. In: Business Process Management (BPM 2023). pp. 20–37. Springer (2023)
6. De Giacomo, G., De Masellis, R., Grasso, M., Maggi, F.M., Montali, M.: Monitoring business metaconstraints based on ltl and ldl for finite traces. In: Business Process Management. Lecture Notes in Computer Science, vol. 8659, pp. 1–17. Springer, Cham (2014)

7. Di Ciccio, C., Maggi, F.M., Montali, M., Mendling, J.: Resolving inconsistencies and redundancies in declarative process models. Information Systems **64**, 425–446 (2017)
8. Donadello, I., Maggi, F.M., Patrizi, F., Tessaris, S., Zorzi, M.: Flexible event log generation using answer set programming. Information Systems **139**, 102714 (2026)
9. Graziosi, R., Ronzani, M., Buliga, A., Francescomarino, C.D., Folino, F., Ghidini, C., Meneghello, F., Pontieri, L.: Generating the traces you need: A conditional generative model for process mining data. In: Proceedings of the 6th International Conference on Process Mining, ICPM 2024. pp. 25–32. IEEE (2024)
10. Käppel, M., Jablonski, S.: Model-agnostic event log augmentation for predictive process monitoring. In: Advanced Information Systems Engineering. pp. 381–397. Springer (2023)
11. López-Pintado, O., Dumas, M., Berx, J.: Discovery, simulation, and optimization of business processes with differentiated resources. Information Systems **120**, 102289 (2024)
12. Maggi, F.M., Bose, R.P.J.C., van der Aalst, W.M.P.: Efficient discovery of understandable declarative process models from event logs. In: Advanced Information Systems Engineering. pp. 270–285. Springer Berlin Heidelberg (2012)
13. Mumuni, A., Mumuni, F.: Data augmentation: A comprehensive survey of modern approaches. Array **16**, 100258 (2022)
14. Serafini, L., d'Avila Garcez, A., Badreddine, S., Donadello, I., Spranger, M., Bhuiyan, A.: Logic tensor networks: Theory and applications. In: Neuro-Symbolic Artificial Intelligence: The State of the Art, Frontiers in Artificial Intelligence and Applications, vol. 342. IOS Press (2021)
15. van Straten, S., Padella, A., Hassani, M.: Leveraging data augmentation and siamese learning for predictive process monitoring. In: 31st International Conference, CoopIS 2025, Marbella, Spain. vol. 15535, pp. 70–87. Springer (2025)
16. Zimmermann, L., Zerbato, F., Weber, B.: What makes life for process mining analysts difficult? a reflection of challenges. Software and Systems Modeling pp. 1–29 (2023)